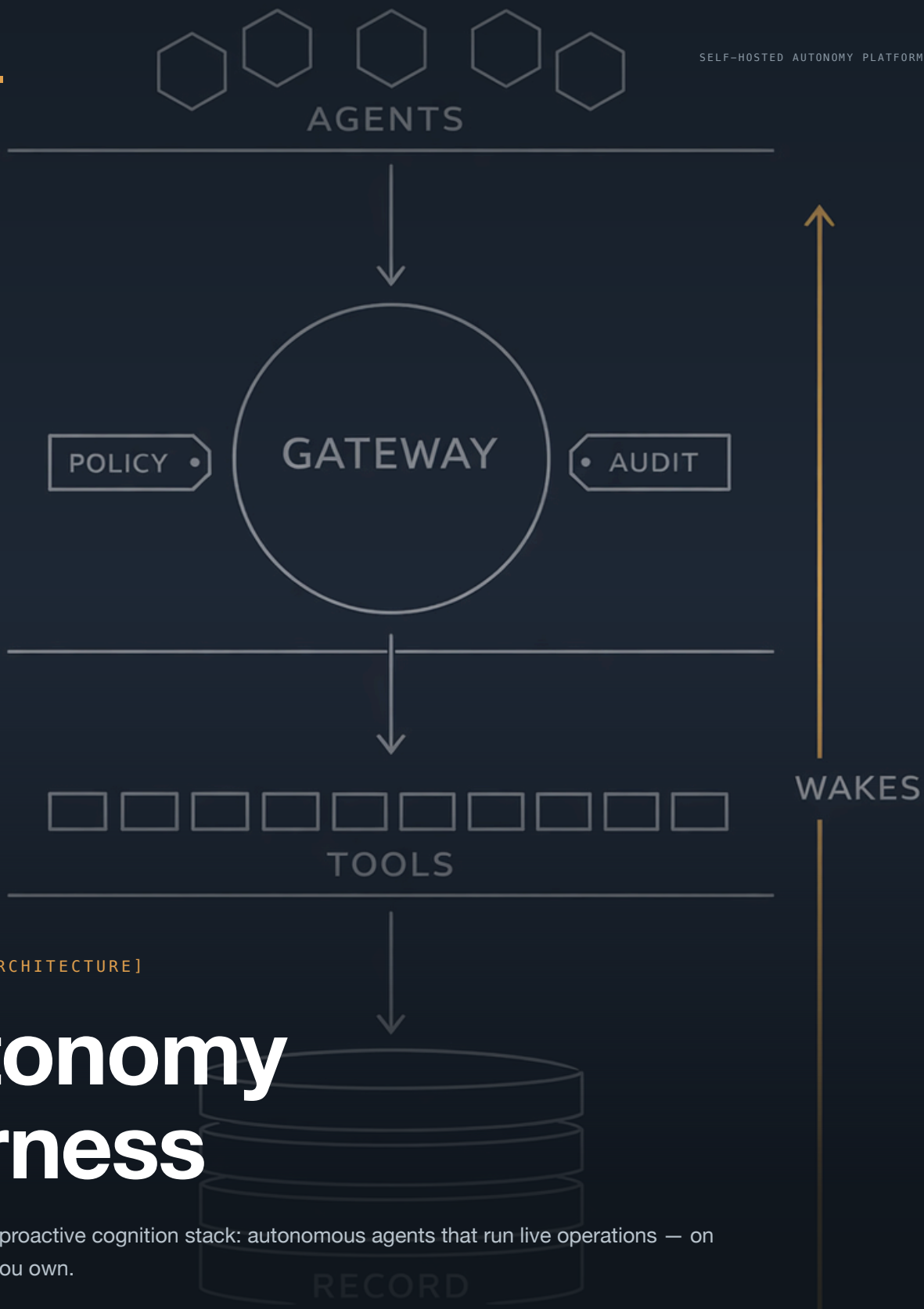




[PLATFORM ARCHITECTURE]

Autonomy Harness

A reactive and proactive cognition stack: autonomous agents that run live operations — on infrastructure you own.

Table of Contents

#	Section
1	Executive Summary
2	Capabilities
3	Data Flow
4	Agent Kernel
5	Edge, Access, and the Gateway
6	Modular Capabilities
7	Durable Tasks and Shared Planes
8	Recording Hub
9	SUMO Showcase
10	Operations and Deployment
11	Verification and Observability
12	Component Reference



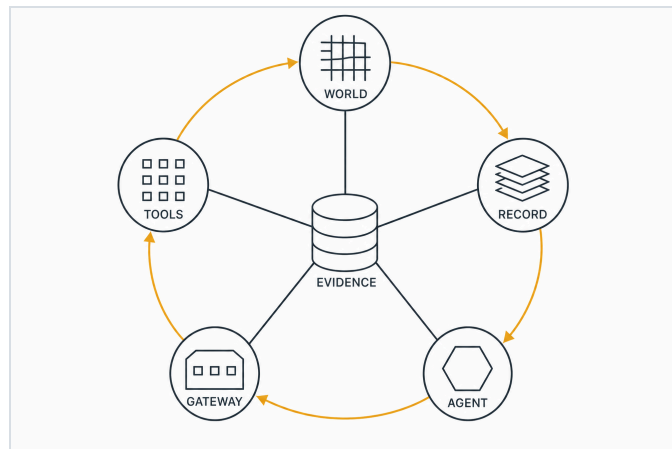
1. Executive Summary

The Autonomy Harness is a reactive and proactive cognition stack: autonomous agents that run live operations on infrastructure the operator owns. Every sensor, simulator, and decision feeds one live operating picture. Agents hunt through it around the clock — they catch conditions the moment they change, decide, and intervene — and between events they drive the mission forward on their own initiative, analyzing, planning, and tasking work under rules of engagement the operator sets.

9	58	~225k
Hosted Servers	Governed Tools	Messages/s, Lossless

Key Findings

- **Cognition is governed from outside.** Every action clears one authenticated door and answers to installation policy, refused by default when in doubt. A model can be wrong, drift, or swallow hostile instructions and still cannot act beyond the envelope.
- **The record is the ground truth and the evidence chain.** Sensors, simulators, and agent decisions stream into one durable record. Any run replays, any moment is queryable, and when someone asks why the system acted, the record answers.
- **Long work carries its own guarantees.** Every long operation is a durable task with a declared recovery class; agents detach, and results wake them within a second of completion — across process death.
- **The installation is sovereign.** Built for contested and disconnected operations: one platform, three installation forms — Compose, Helm, and a verified offline bundle with provenance included.
- **A live city proves the loop.** A pinned SUMO traffic world drives perceive → decide → act end to end through the same contracts as every other capability.



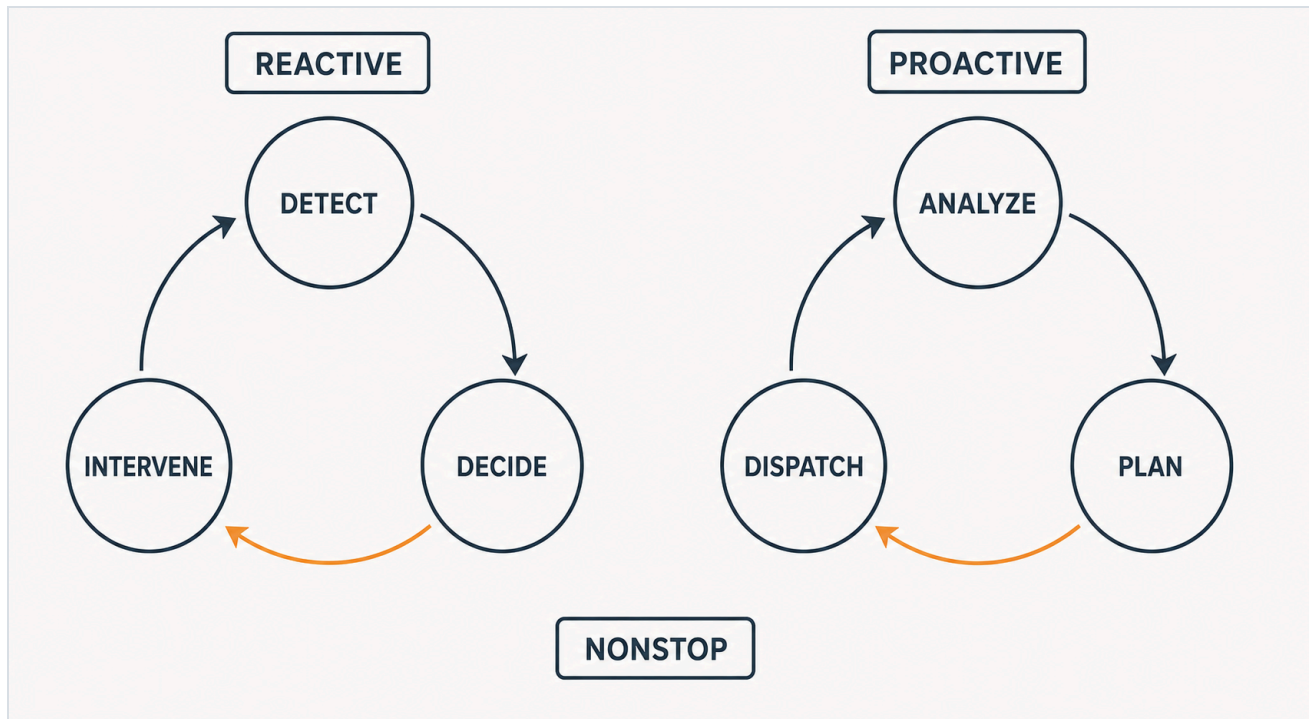
THE WHOLE LOOP: THE WORLD IS RECORDED, AGENTS PERCEIVE AND ACT THROUGH GOVERNED TOOLS, EFFECTS LAND BACK IN THE WORLD — AND EVERY STEP LEAVES EVIDENCE.

[SCOPE]

This document describes the architecture and live, smoke-tested behavior of the current installation: the agent kernel, the governed tool plane, the durable record, and the deployment forms. Each installation is owned and operated by the organization that deploys it.



2. Capabilities



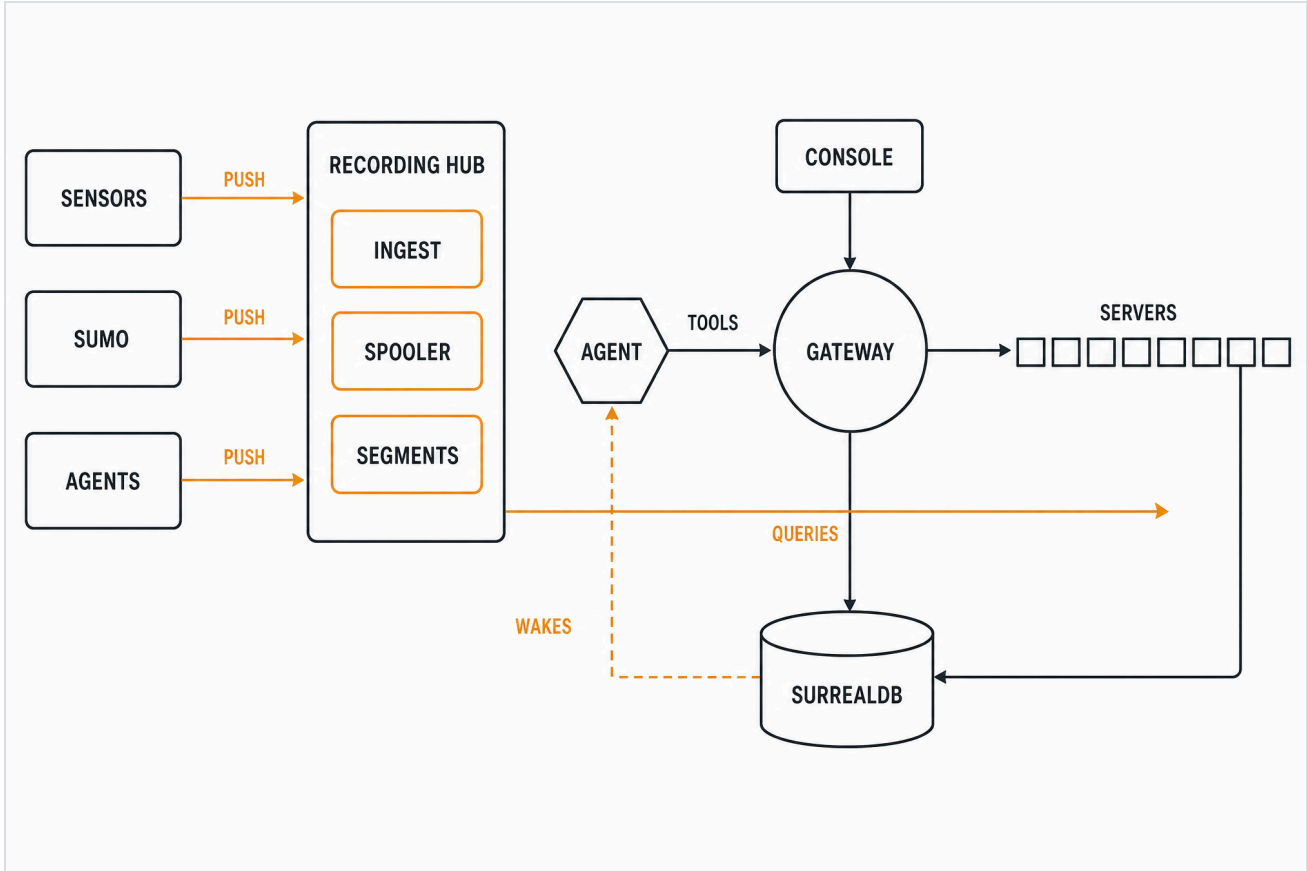
REACTIVE AND PROACTIVE COGNITION: REACT THE MOMENT CONDITIONS CHANGE; DRIVE THE MISSION FORWARD BETWEEN EVENTS — NONSTOP.

- **Detect, decide, intervene — nonstop.** Agents chase the live picture around the clock: they catch conditions the moment they change, decide, and intervene — retiming signals, rerouting vehicles, retasking work — and between events they drive the mission forward on their own initiative. The tempo holds through restarts, upgrades, and machine failures, memory and in-flight work intact.
- **Dispatch long work and move on.** A render, a forecast, a plan, or a 500-step simulation batch carries its own guarantees: it completes on its own, delivers the result the moment it is ready, and an interruption has a defined, recorded outcome.
- **Hold a complete operating picture.** Every sensor, simulator, and agent decision streams into one durable record at ~225k messages/s, lossless — ground truth for agents in the moment, an evidence chain after the fact.
- **Set the rules of engagement.** Every action clears one authenticated door and answers to your policy — who is acting, on which tenant's data, at what sensitivity — refused by default when in doubt. Agents act freely inside the rules, and the rules are yours.
- **Give agents real capabilities.** Generate media, run any SQL safely in its own sandbox, forecast time series, solve planning problems, transform coordinates, render charts, and drive a live viewer — all under one task, artifact, and policy contract. Adding a capability, at the edge or from a remote server, is a registration — the platform stays put.
- **Share results deliberately.** Every output is an owned artifact. Grant read, write, or admin to users and groups; mark an artifact releasable and mint a revocable, expiring link. Downloads pass policy and audit before bytes move.
- **Command from one console.** The operations console opens on the running installation: health, tasks, artifacts, agents, recordings, the tool network, policy, and audit evidence.
- **Own the whole installation.** Built for contested and disconnected operations — your hostname, your identity provider, your object store — on one host, in a cluster, or fully disconnected.



3. Data Flow

The decision cycle, engineered. **Observe:** producers push into the Recording Hub, and agents query the record back out through governed tools. **Act:** agents and operators call tools through the gateway; long work becomes a durable task whose completion wakes the sleeping agent. **Account:** every auth decision, tool call, task transition, artifact grant, and agent step lands in the platform store and the decision log.



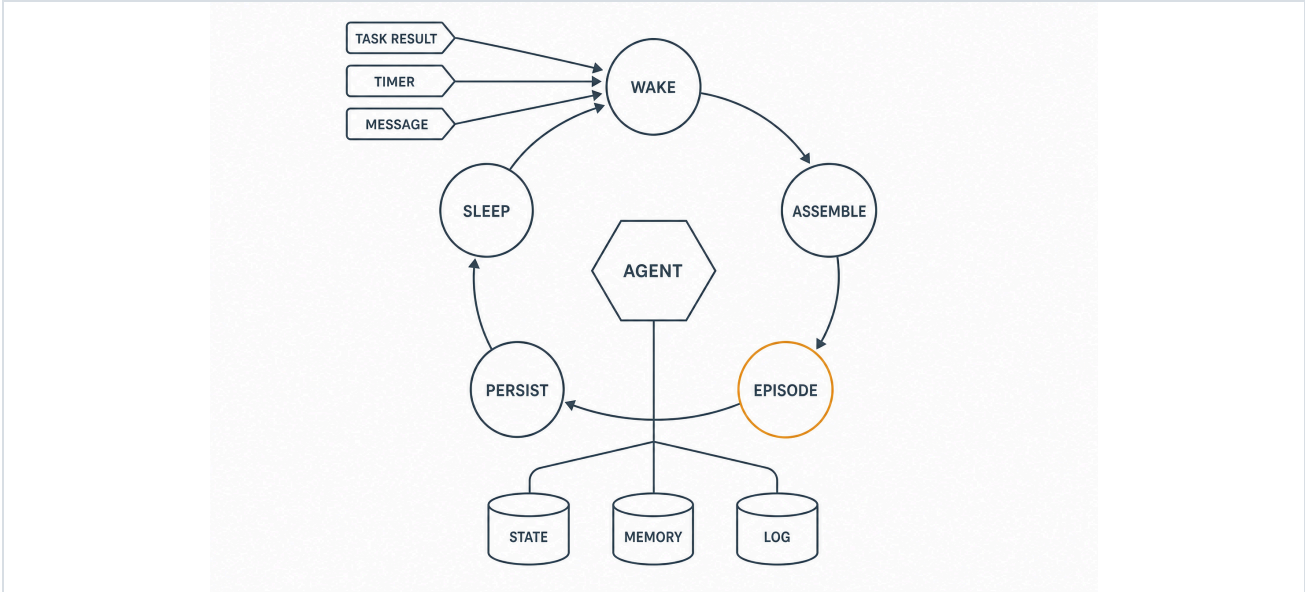
THE WHOLE SYSTEM: FILES ARE THE RECORD · THE HUB IS THE BUS · THE GATEWAY IS THE DOOR · SURREALDB IS THE PLATFORM'S MEMORY.

Flow	Path	Guarantee
Observe	Producers push typed Rerun streams to one ingest point; segments become governed catalog records; agents read slices through the recording server.	The hub persists first; a crash mid-write leaves readable files.
Act	Every tool call enters through the gateway under one flat namespace; quick tools answer inline; long tools return a durable task handle immediately.	Task completion pushes a wake; the next episode assembles with the result.
Account	Auth, policy, admin, and task transitions write audit evidence; every decision records what the agent saw, chose, and why.	Usage records meter provider spend and tool activity per principal.



4. Agent Kernel

An agent runs forever because durable stores are the memory: every episode's context is assembled fresh from the agent's memory planes, and everything in flight survives process death. The kernel executes typed manifests — an agent type is a manifest, a preamble, and SQL migrations.



WAKE → ASSEMBLE → EPISODE → PERSIST → SLEEP, FOREVER.

Stage	What happens
Wake	A batch of wakes, coalesced and deduped; every wake is persisted with its disposition, so the ledger explains why each episode ran.
Assemble	State header, wake payload, and manifest-defined SQL sections over the memory database, under a token budget.
Episode	One bounded LLM run. Quick gateway tools answer inline; long tools dispatch as durable tasks. Hooks record every step and terminate over-budget runs.
Persist	Unresolved task descriptors land in the platform store; the episode closes with usage counters and a summary.
Sleep	Idle on the wake bus. A leased watcher per detached task waits on the result; the heartbeat bounds silence.

Wakes: task result · resource change · timer · operator message · elicitation answer. A wake survives process death — descriptors and watcher leases rehydrate from the platform store on any boot, so kill and restart resumes the same tasks.



4.1 Memory Planes and Durability

Memory planes

- **Scheduling truth.** The SurrealDB platform store owns episodes, wakes, task watchers, and leases; the outbox and changefeed deliver wakes across processes.
- **Analytical memory.** A sandboxed DuckDB per agent: domain tables (the Pilot: targets, missions, waypoints, constraints, beliefs), a kv store, and an episode-log projection for context assembly.
- **Decision log.** An append-only Rerun recording: every turn, tool call, task lifecycle, prompt, and summary — time-indexed and watchable live.
- The agent reads all three itself: `memory_query` (guarded SQL), `memory_write` (typed mutations), and `timeline_query` (dataframe reads over the log).

Durability, proven

- Detach at episode end: a task's descriptor outlives the process; any later boot rehydrates the live handle and keeps waiting.
- Token rotation is make-before-break; task ids are principal-scoped, so continuity holds across reconnects.
- The wake is push, arithmetically: with polling pinned at 60s and the heartbeat at 600s, the smoke's agent slept the task's full ~15s and woke within a second of completion — verified live with a real model end to end.
- Budget hooks book overruns as `budget_terminated`, a first-class outcome; hourly windows defer non-priority wakes.

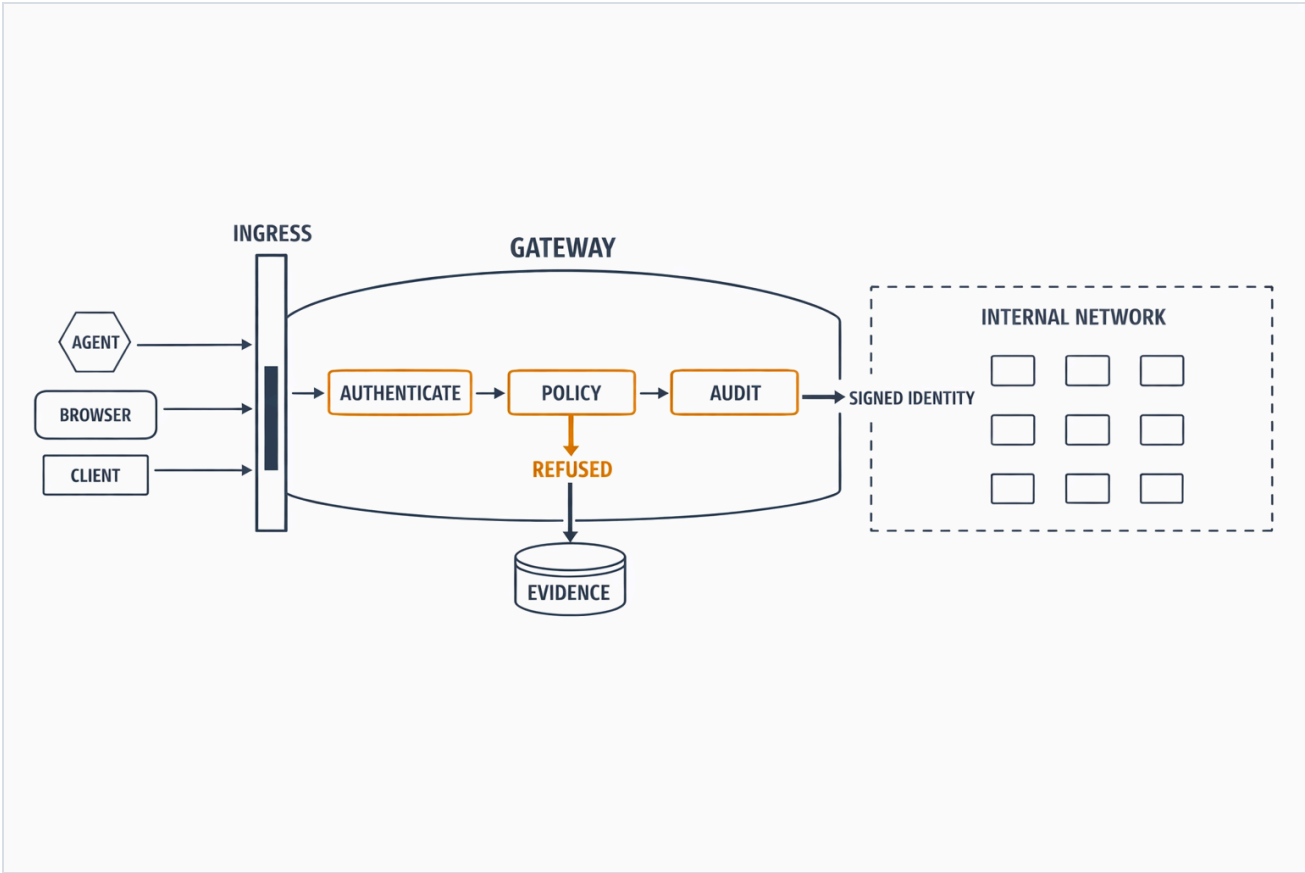
[PRINCIPLE]

Agents are data. A typed manifest — model endpoint, gateway principal, episode limits, budgets, schedule, context SQL sections, writable tables — is validated fail-closed at boot. A preamble and domain migrations complete the profile; cross-episode instructions live there or in memory.



5. Edge, Access, and the Gateway

Every client enters through the installation's own ingress: the Caddy edge on Compose, an Ingress on Helm. /mcp, /oauth, /admin, and /artifacts reach the gateway; /console and /auth reach the console BFF; /s/{token} reaches the artifact service for public links. Every hosted server lives on the internal network and requires a gateway-signed Ed25519 identity assertion.



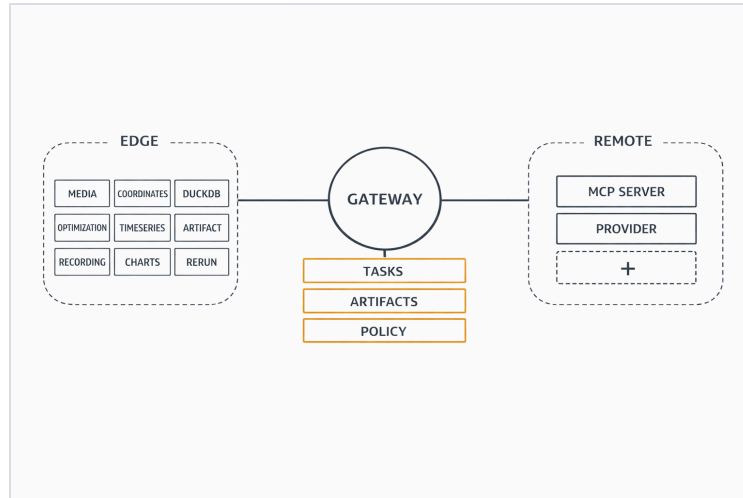
EVERY ACTION RUNS THE GAUNTLET: ONE DOOR IN, AUTHENTICATE → POLICY → AUDIT, REFUSED BY DEFAULT, SIGNED IDENTITY ONWARD, EVIDENCE KEPT.

Function	Behavior
Authentication	Browser OAuth with PKCE, client credentials with private-key JWT, identity-assertion grant (ID-JAG) exchange, JWKS, token issuance and revocation, durable refresh rotation — reuse revokes the family.
Policy & audit	Typed ABAC/RBAC over tools, resources, tasks, and artifacts; tenant and data-label checks for regulated data; fail-closed decisions with reason codes; a durable audit trail for admin, auth, and policy operations.
Control plane	Validated server / profile / policy catalog, authenticated live apply and reload, secret references with redaction, an admin API for the console, and policy-checked, audited artifact downloads.
Protocol surface	Tools, resources and templates, prompts, completions, tasks, subscriptions, notifications, artifacts, and usage — forwarded and aggregated under one flat tool namespace (media__run, duckdb__query).



6. Modular Capabilities

Capability is modular. Nine modules run at the edge today, inside the installation, behind gateway-signed identity — and the surface is open: a new capability, whether another edge server, a stdio server carried by the bridge, or a **remote MCP endpoint**, registers in the validated catalog, applies live, and immediately answers to the same tasks, artifacts, and policy contract. The gateway projects every module into one flat tool menu.



EDGE AND REMOTE CAPABILITY MODULES — ONE CONTRACT GOVERNS ALL, AND THE NEXT SLOT IS OPEN.

Module	Capability	Tools
media	Provider-backed generative media with webhook-driven completion, a model catalog, and pricing.	<code>run</code> · <code>models</code> · <code>model_schema</code> · <code>artifact</code>
coordinates	Geodesy and reference-frame math: WGS84 / ECEF / ENU / NED, projected CRS, geofences.	<code>convert_frame</code> · <code>transform_crs</code> · <code>derive_local_frame</code> · <code>geodesic_inverse</code> · <code>geodesic_direct</code> · <code>validate_geofence</code> · <code>batch_transform</code>
duckdb	Arbitrary SQL over principal-owned databases inside a locked, resource-bounded sandbox.	<code>query</code> · <code>execute</code> · <code>ingest</code> · <code>export</code>
optimization	Task-option planning for one or many agents over typed objects or SQL-readable option rows.	<code>plan</code>
timeseries	Forecasting over any SQL-readable time-series source, visualized as Rerun recordings.	<code>forecast</code>
artifact	Discovery, metadata, grants, release, and sharing — the governance surface over the byte store.	<code>metadata</code> · <code>grant_access</code> · <code>revoke_access</code> · <code>set_release_state</code> · <code>create_share_link</code> · <code>revoke_share_link</code>
recording	Authorized access to the Recording Hub: discovery, dataframe queries, subscriptions, publication.	<code>query_recording</code> · <code>seal_recording</code>
charts	Chart rendering projected through the gateway, turning query results into shareable visuals.	<code>render_chart</code> · <code>compile_chart</code> · <code>validate_chart</code> · <code>list_chart_types</code> · <code>create_chart_view</code>
rerun	A live Rerun viewer driven entirely over MCP — session, capture, inspection, and UI automation.	17 interactive tools (<code>connect ... batch</code>)

7. Durable Tasks and Shared Planes

Shared durable tasks

Every long-running tool runs on the shared task runtime and the final Veoveo task extension. Task IDs are UUIDv7; creation, leases, transitions, results, and outbox events are atomic in the platform store. Recovery is declared per operation:

Class	Meaning
<code>resume</code>	Deterministic, side-effect-safe work may be reclaimed after its lease expires.
<code>webhook_wait</code>	A durably submitted provider job waits for its signed webhook. Completion is webhook-only.
<code>interrupted_indeterminate</code>	Interrupted mutating work fails closed and is never replayed automatically.

Shared planes

- **Platform store.** One SurrealDB 3.2.0 node (RocksDB) is the durable coordination authority: identity, policy, control-plane revisions, tasks, artifacts, recordings, agents, wakes, audit evidence, and usage. The transactional outbox and replayable changefeed carry all cross-process work.
- **Artifact plane.** A single byte-level policy-enforcement point. S3-compatible object store for bytes; the platform store owns grants, labels, and release state. Fresh opaque identity per occurrence (`artifact://{uuidv7}`); hashes provide integrity and tenant-local dedup. `artifact://` references let one server consume another's output.
- **Contract core.** The typed, provider-neutral contract every server shares, validated fail-closed at startup: strong ID newtypes, deployment profiles, shared task and artifact schemas, and gateway-to-server Ed25519 identity assertions — servers hold public verification keys only.

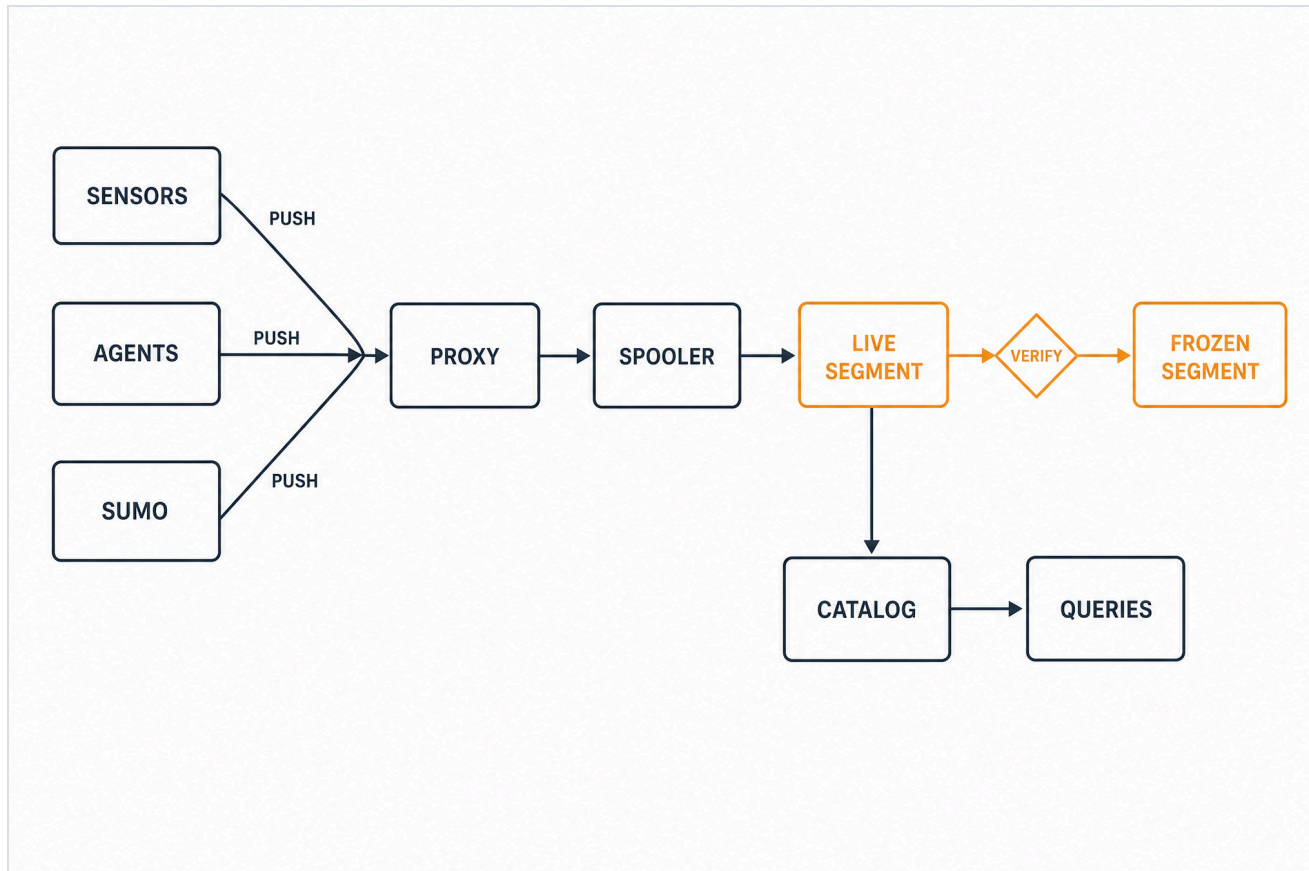
[PRINCIPLE]

Two sharing modes, both explicit: authorized grants to users or groups, still constrained by tenant and label policy; and read-only anyone-with-link bearers for artifacts explicitly marked releasable — expiring, download-limited, and revocable.



8. Recording Hub

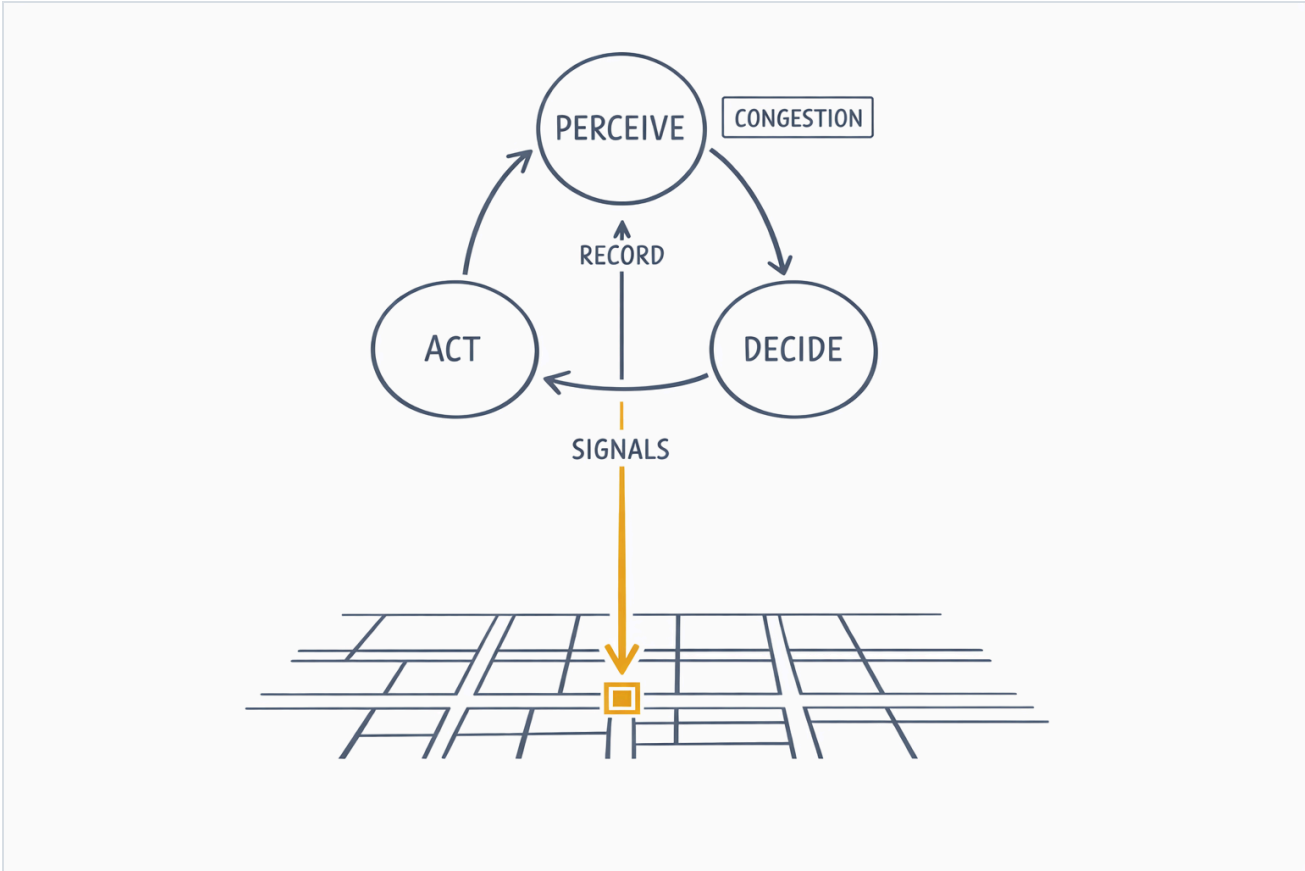
Every sensor, estimator, and agent streams Rerun data into one gRPC ingest point. A spooler embeds the proxy and persists every message — fsynced, to day-partitioned segment files — so the write is the first thing that happens to arriving data. Files are footer-less while live (crash-decodable) and are verified before their optimized replacement at freeze. Every recording and segment becomes a governed record in the platform store, and the recording server answers authorized queries over the same bytes.



DURABLE CAPTURE: KILL -9 LEAVES A READABLE SEGMENT · RESTART RESUMES INTO A .RN SIBLING · ~225K MSGS/S LOSSLESS.

9. SUMO Showcase

The SUMO simulator runs the pinned, MIT-licensed LuST Luxembourg scenario in its own container. A Rust `sumo-mcp` server owns the single serialized TraCI connection, pushes the world into the hub as typed `/world/sumo/**` streams, and exposes SUMO control as governed `sumo__*` tools. The long operations are durable tasks the agent detaches from and wakes on — the same sleep/wake the kernel already runs, now driven by a real simulator.



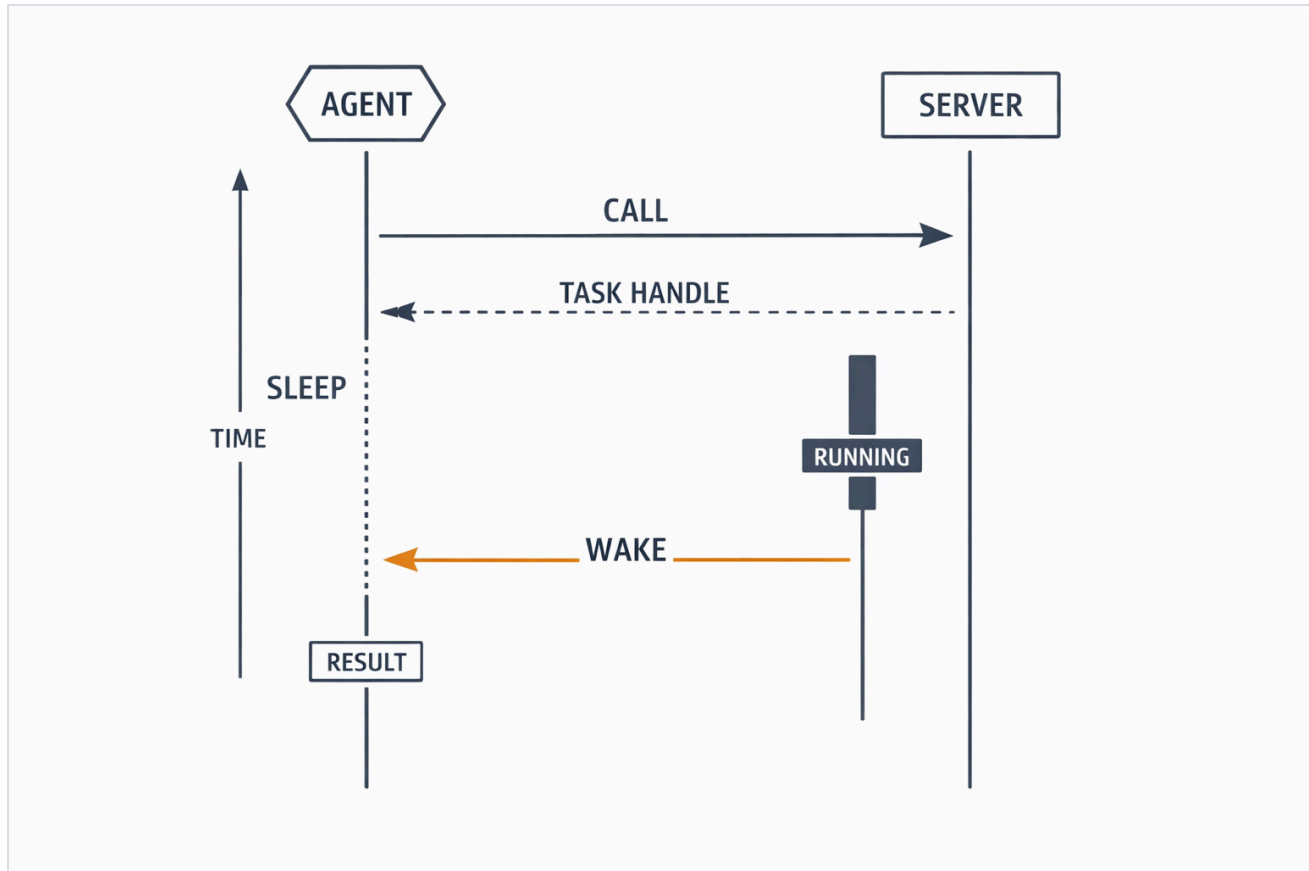
A LIVE TRAFFIC WORLD PROVING PERCEIVE → DECIDE → ACT, END TO END.

Surface	Detail
Sync tools	<code>query_state</code> · <code>describe_scenario</code> · <code>set_signal_phase</code> · <code>reroute_vehicle</code> · <code>set_edge_speed</code> · <code>close_lane</code> · <code>open_lane</code>
Durable tasks	<code>run_batch</code> (live sim — interrupted_indeterminate) · <code>generate_network</code> · <code>compute_routes</code> · <code>optimize_signals</code> (real SUMO CLIs, resumable)
Events	<code>sumo://congestion</code> is subscribable — a jam forming is a wake the moment it happens.
Outputs	Successful task outputs upload through a task-bound write capability to the shared artifact plane.



9.1 Task Sleep/Wake

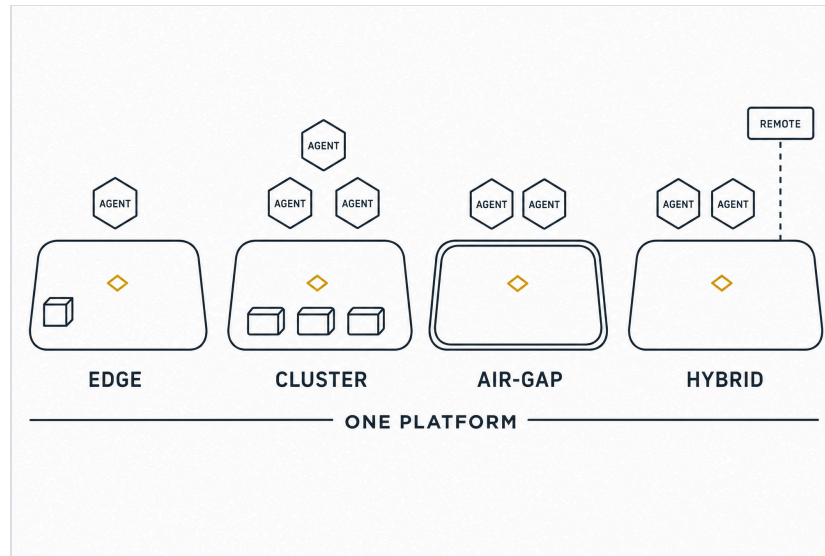
The exact detach → sleep → wake path, on the final task extension the gateway and kernel speak: the agent calls a task-augmented tool, receives a durable handle immediately, persists the descriptor, ends the episode, and sleeps. The server runs on; completion pushes a wake, and a fresh episode assembles with the result — within a second, across process death.



DETACH → SLEEP → WAKE: ZERO CONNECTIONS HELD, ZERO EPISODES BURNED WAITING.

10. Operations and Deployment

Deployment is a spectrum, and cognition is constant across it. The same platform runs at the operation's edge on one host, across a cluster, sealed inside an air gap, or hybrid — an edge installation that reaches remote capability when links allow. Wherever it lands, it carries the full stack: **one agent or a team of them, every capability module, and the operational intelligence to run the mission** — the console, the record, and the audit trail.



SAME PLATFORM, SAME COGNITION — EDGE, CLUSTER, AIR-GAP, OR HYBRID; ONLY HYBRID REACHES OUT, AND ON YOUR TERMS.

Form Shape

Edge	One host at the operation: just compose-up brings up the full stack behind a loopback Caddy edge. Public exposure is the operator's ingress decision.
Cluster	The Helm form: one SurrealDB StatefulSet, default-deny NetworkPolicy, optional service-mesh mTLS, separate migration and runtime credentials, operator-supplied telemetry and SIEM wiring.
Air-gap	The verified offline bundle: pinned images, Compose and Helm material, config schemas, checksums, and SBOMs. Installation and verification complete entirely inside the boundary.
Hybrid	An edge installation that registers remote MCP endpoints and provider-backed modules. The reach is governed like everything else — same policy, same audit, same task contract.

Operational intelligence, wherever it lands

- **Command from the console.** The first screen is the live installation — health, tasks, artifacts, agents, recordings, policy, audit — with cancellation and sharing as one-click operations.
- **Cognition scales in place.** Agents are data: field another agent by adding a manifest. The same wake bus, budgets, and memory planes run one agent or a team.
- **Evidence deploys with the platform.** The record, the audit trail, and usage metering are part of every form — air-gapped included.

11. Verification and Observability

- **Conformance CLI.** A generic MCP client exercising the full protocol surface — info, resources, prompts, completions, runs, usage, artifacts — plus auth-metadata discovery and token tooling.
- **Smoke harness.** Rust end-to-end scenarios with process orchestration: fake IdP, provider, and upstream services; a real Keycloak realm; live agent-kernel sleep/wake; and the full SUMO world in a disposable Compose project.
- **CI.** Formatting, clippy, tests, the console UI build, the Keycloak integration path, and the deployment contract checks run on every change.
- **Telemetry.** OpenTelemetry traces and logs across gateway and servers, with audit summaries as durable evidence for regulated deployments.

[EVIDENCE]

Every capability in this document is live, smoke-tested behavior of the current installation. The record replays, the audit trail is durable, and the decision log carries each agent's rationale — when someone asks why the system acted, the record answers.



12. Component Reference

Component	Role	What it is
ingress	ours	The single public door, owned by the operator: Caddy on Compose, an Ingress on Helm. One place to defend and observe.
mcp-gateway	ours	The front desk for tools: authenticates the caller, checks policy, projects every server's tools into one flat menu, writes the audit trail.
console	ours	The cockpit: a React UI plus a Rust BFF that owns browser login, encrypted sessions, and CSRF enforcement.
agent-kernel	ours	The runtime for long-lived agents: short episodes, durable memory, wake on a result, a timer, or a message.
platform store	OSS	SurrealDB 3.2.0 — the platform's memory: identity, policy, tasks, artifacts, recordings, agents, audit, and the transactional outbox.
task runtime	ours	The ledger for long work: UUIDv7 task ids with atomic leases, transitions, results, and declared recovery per operation.
rerun proxy	OSS	The ingest socket: a gRPC server every producer pushes into, rebroadcast live to anyone reading.
hub-spooler	ours	Catch-and-persist: embeds the proxy and fsyncs every message to disk before anything else.
.rrd segment	format	The record on disk: one growing file per dataset / day / recording, readable even mid-write.
catalog	ours	The card catalog: typed installation identity for every recording and segment; startup reconciles crash leftovers.
recording server	ours	The reading room: authorized discovery, latest-at / range / dataframe queries, subscriptions, and sealing to an artifact.
SUMO / TraCI	OSS	The traffic simulator and its control socket — a live world producing positions and states to record and reason about.
sumo-mcp	ours	The bridge: a Rust service holding the one serialized TraCI connection, pushing the world into the hub, exposing governed control.
MCP	protocol	The tool language every server speaks — which is why one gateway can front them all with one set of rules.
durable task	concept	A tool call you walk away from: the handle returns immediately; the row, lease, and result live in the platform store.
resource · subscribe	concept	A condition that wakes you: subscribing to a resource means the agent is notified the moment it changes.

